Vel Tech High Tech

Dr.Rangarajan Dr.Sakunthala Engineering College

(An Autonomous Institution Affiliated to Anna University,Chennai)

Approved by AICTE, Affiliated to Anna University. `A` Grade Accrediation by NAAC.

No.60, Avadi – Vel Tech Road, Chennai-600 062.

AD8511 – MACHINE LEARNING LABORATORY

NAME :

REGISTER NO. :

ROLLNO :

BRANCH : B.Tech. ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

YEAR : III

SEMESTER : V

Vel Tech High Tech

Dr.Rangarajan Dr.Sakunthala Engineering College

(An Autonomous Institution Affiliated to Anna University,Chennai)
Approved by AICTE, Affiliated to Anna University. `A` Grade Accrediation by NAAC.
No.60, Avadi – Vel Tech Road, Chennai-600 062.

BONAFIDE CERTIFICATE

Name_____

Year:____III_____Semester:_____V____Branch: B.Tech – Artificial Intelligence and Data Science

University Register No: [ ] **College Roll No:** [ ]

Certified that this is the bonafide record of work done by the above student in A D 8 5 1 1 **--MACHINE LEARNING LABORATORY** during the academic year **2022-2023.**

Signature of Course Incharge **Signature of Head of the Department**

Submitted for the University Practical Examination held on..... at **VEL TECH HIGH TECH Dr.RANGARAJAN Dr.SAKUNTHALA ENGINEERING COLLEGE, No.60, AVADI – VEL TECH ROAD, AVADI, CHENNAI-600062.**

Signature of Examiners

Internal Examiner:…… **External Examiner:……..……**

Date:……..……..

VEL TECH HIGH TECH DR.RANGARAJAN DR.SAKUNTHALA ENGINEERING COLLEGE

INSTITUTION VISION

Pursuit of excellence in technical education to create civic responsibility with competency.

INSTITUTION MISSION

1. To impart the attributes of global engineers to face industrial challenges with social relevance.
2. To indoctrinate as front runners through moral practices. 3. To attain the skills through lifelong learning.

DEPARTMENT VISION

- To be a centre of excellence in the field of Artificial Intelligence and Data Science.

DEPARTMENT MISSION

- To provide conducive learning environment for quality education in the field of Artificial Intelligence and Data Science.
- To pursue industry institute interaction and promote collaborative research activities.
- To empower the students with ethical values and social responsibilities in their profession.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO 1:

Excel in professional Career with technical skills to build intelligent system and applications in the area of Artificial Intelligence opportunities. and Data Science to identify raw opportunities.

PEO 2:

Shall posses critical thinking, professional skills and Strong work ethics to solve real Problems required to cater the needs of industry and to serve the society.

PEO 3:

To enable graduates to think logically, persue life long learning and collaborate with an ethical attitude in a multi-disciplinary team

PROGRAMME SPECIFIC OUTCOMES (PSOs)

By the time of graduation, the undergraduate Artificial intelligence and Data science students can have the ability of

PSO's	PROGRAMME SPECIFIC OUTCOMES (PSOs)
PSO1	Design and Analyze optimal artificial intelligence solutions to real-world problems in data analysis / augmented reality / virtual reality / robotics.
PSO2	Apply data science techniques and project development skills to provide solutions for complex problems.
PSO3	Exhibit ethical attitude and social responsibility in their profession.

PROGRAM OUTCOMES:

1. **Engineering knowledge**: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis**: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions**: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems**: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage**: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society**: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability**: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics**: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work**: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication**: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. **Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multi disciplinary environments.
12. **Life Long Learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

AD8511 MACHINE LEARNING LABORATORY L T P C
0 0 4 2

OBJECTIVES:

- To get practical knowledge on implementing machine learning algorithms in real time
- problem for getting solutions
- To implement supervised learning and their applications
- To understand unsupervised learning like clustering and EM algorithms
- To understand the theoretical and practical aspects of probabilistic graphical models.

LIST OF EXPERIMENTS:

- Implement the concept of decision trees with suitable data set from real world problem and classify the data set to produce new sample.
- Detecting Spam mails using Support vector machine
- Implement facial recognition application with artificial neural network
- Study and implement amazon toolkit: Sagemaker
- Implement character recognition using Multilayer Perceptron
- Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs.
- Implement sentiment analysis using random forest optimization algorithm
- Write a program to construct a Bayesian network considering medical data. Use this model to demonstrate the diagnosis of heart patients using standard Heart Disease Data Set. You can use Java/Python ML library classes/API.
- Choose best machine learning algorithm to implement online fraud detection
- Mini-project: students work in team on any socially relevant problem that needs a machine learning based solution, and evaluate the model performance.

TOTAL : 60 PERIODS

COURSE OUTCOMES:

At the end of the course, the student will be able to:

CO No	Course Outcomes	Knowledge Level
C507.1	***Understand*** various machine learning techniques for the real world problem.	K3
C507.2	***Implement*** artificial neural Network and amazon toolkit: Sagemaker.	K3
C507.3	***Analyze*** various algorithms for sentiment detection.	K3
C507.4	***Implement*** a prediction model for diagnosing heart disease and online fraud detection .	K3
C507.5	***Apply*** suitable Machine learning techniques to solve various computing problems.	K3

CO-PO MAPPING:

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO 10	PO 11	PO 12	PSO1	PSO2
C507.1	2	1	1	1	3	-	-	-	2	2	-	-	3	3
C507.2	3	3	2	1	3	-	-	-	2	2	-	-	3	3
C507.3	3	3	3	2	3	-	-	-	2	2	-	-	3	3
C507.4	3	3	2	1	3	-	-	-	2	2	-	-	3	3
C507.5	3	3	2	1	3	-	-	-	2	2	-	-	3	3
C507	**2.8**	**2.6**	**2**	**1.2**	**3**	**-**	**-**	**-**	**2**	**2**	**-**	**-**	**3**	**3**

TABLE OF CONTENTS

S.No	DATE	NAME OF THE EXPERIMENT	PAGE NO	MARKS	SIGNATURE
1		Implement the concept of decision trees with suitable data set from real world problem and classify the data set to produce new sample.			
2		Detecting Spam mails using Support vector machine			
3		Implement facial recognition application with artificial neural network			
4		Study and implement amazon toolkit: Sagemaker			
5		Implement character recognition using Multilayer Perceptron			
6		Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs.			
7		Implement sentiment analysis using random forest optimization algorithm			
8		Write a program to construct a Bayesian network considering medical data. Use this model to demonstrate the diagnosis of heart patients using standard Heart Disease Data Set. You can use Java/Python ML library classes/API.			
9		Choose best machine learning algorithm to implement online fraud detection			
10		Mini-project: students work in team on any socially relevant problem that needs a machine learning based solution, and evaluate the model performance.			

Ex. No.: 01	Implement the concept of decision trees with suitable data set from real-world problem and classify the data set to produce new sample.
Date:	

AIM:

To implement the concept of decision trees with suitable data sets from real-world problems and classify the data set to produce new samples.

ALGORITHM:

Step 1: Start the program

Step 2: Import python modules such as Numpy, Pandas, Scikitlearn.

Step 3: Load the Dataset using pandas read_csv()

Step 4: Train the decision tree.

Step 5: Plot the regression using matplotlib.

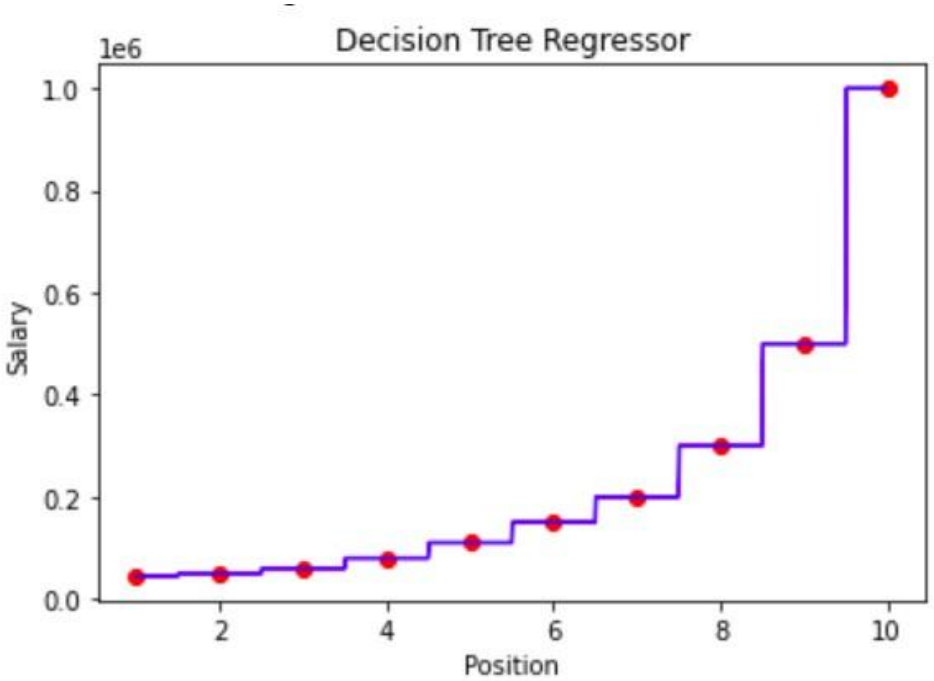
Step 6: Print the predicted salary.

Step 7: Stop the program.

PROGRAM:

```
# Step 1 - Load Dataset
import pandas as pd
import numpy as np
from sklearn.tree import DecisionTreeRegressor
import matplotlib.pyplot as plt

# Step 2 - Load Dataset
dataset = pd.read_csv("Position_Salaries.csv")
X = dataset.iloc[:, 1:2].values
y = dataset.iloc[:, 2].values

# Step 3 - Fit Decision Tree Regressor
regressor = DecisionTreeRegressor(criterion="mse")regressor.fit(X,
y)

# Step 4 - Visualize
X_grid = np.arange(min(X), max(X), 0.01)
X_grid = X_grid.reshape((len(X_grid),1))

plt.scatter(X, y, color="red")
plt.plot(X_grid, regressor.predict(X_grid), color="blue")
plt.title("Decision Tree Regressor")
plt.xlabel("Position")
plt.ylabel("Salary")
plt.show()

# Step 5 - Predict
y_pred = regressor.predict([[6.5]])
print('The predicted salary of a person at 6.5 Level is
',y_pred)
```

OUTPUT:




```
The predicted salary of a person at 6.5 Level is  [150000.]
```

RESULT:

Thus the implementation of the concept of decision trees with suitable data sets from real-world problems and the classification of the data set to produce new sample was successfully executed

Ex. No.: 02	Detecting Spam mails using Support vector machine
Date:	

AIM:

To implement the concept of Support vector machine for detecting Spam mails.

ALGORITHM:

Step 1: Start the program

Step 2: Importing the necessary modules.

Step 3: Read the dataset using pandas read_csv() method.

Step 4: Split the test and train dataset.

Step 5: convert it into a vector.

Step 6: Fit the model using SVC.fit()

Step 7: Print the score of the classifier.

Step 8: Stop the program

PROGRAM:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import GridSearchCV
from sklearn import svm

data = pd.read_csv('spam.csv')
data.head()
data.info()
data.describe()
data.isnull()

X = data['EmailText'].values
y = data['Label'].values

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split( X, y,
test_size=0.2, random_state=0)

# Converting String to Integer
cv = CountVectorizer()
X_train = cv.fit_transform(X_train)
X_test = cv.transform(X_test)

print(X_train)
print(X_test)

from sklearn.svm import SVC
classifier = SVC(kernel = 'rbf', random_state = 10)
classifier.fit(X_train, y_train)
print(classifier.score(X_test,y_test))
```

OUTPUT:

[14]:
```
from sklearn.svm import SVC
classifier = SVC(kernel = 'rbf', random_state = 10)
classifier.fit(X_train, y_train)
print(classifier.score(X_test,y_test))
```

```
0.9766816143497757
```

RESULT:

Thus the implementation of the concept of Support vector machine for detecting spam mails was executed successfully.

Ex. No.: 03	Implement facial recognition application with artificial neural network
Date:	

AIM:

To Implement facial recognition application with artificial neural network.

ALGORITHM:

Step 1: Start the program

Step 2: Importing the necessary modules.

Step 3: Read the image dataset using os module.

Step 4: Read the image using cv2.imread() method.

Step 5: Convert image BGR to RGB using cv2.cvtColor() method.

Step 6: Predict the image using mp.solutions.face_detection.process() method.

Step 7: Print the output using matplotlib.pyplot.imshow() method.

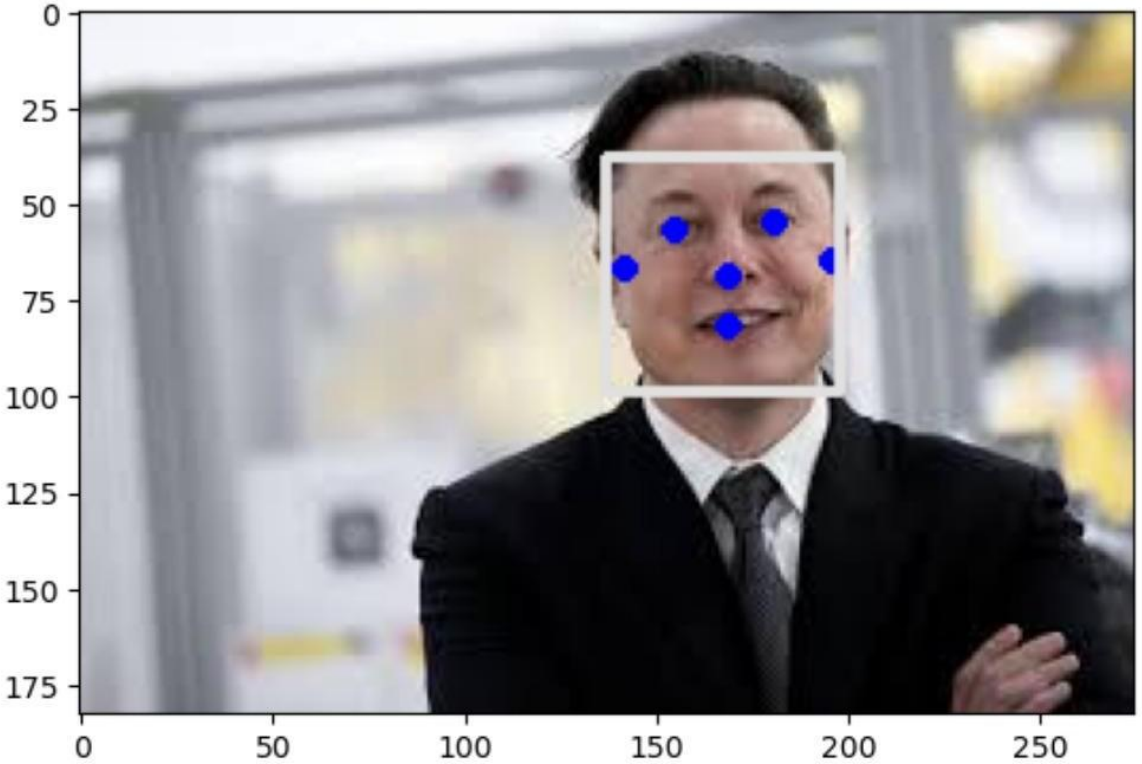
Step 8: Stop the program

PROGRAM:

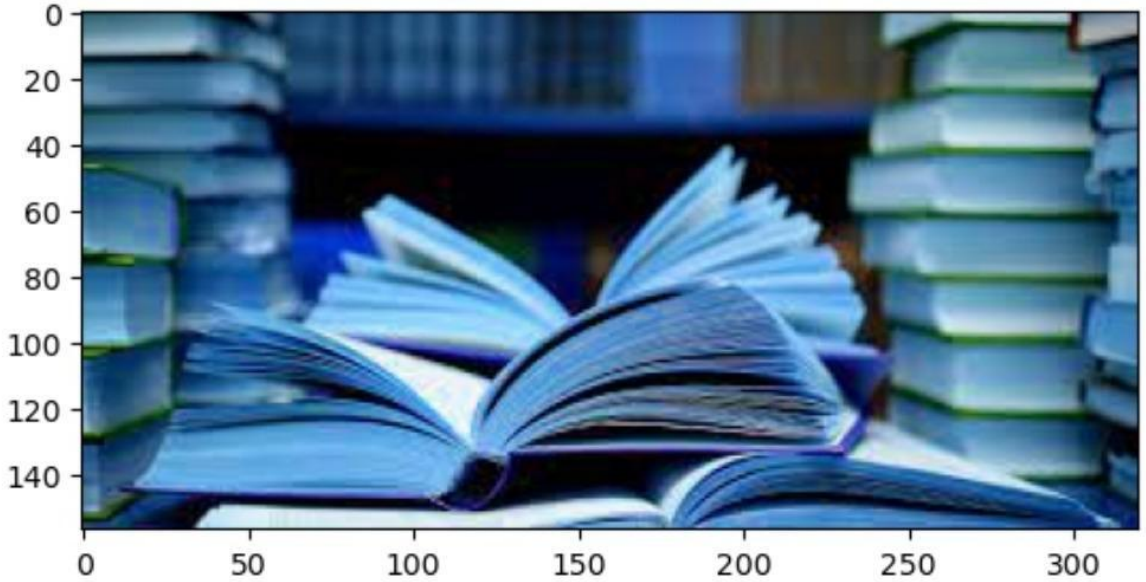
```
import cv2
import mediapipe as mp
import numpy as np
import os
from IPython.display import Image
import matplotlib.pyplot as plt

mp_face_detection = mp.solutions.face_detection
mp_drawing = mp.solutions.drawing_utils

IMAGE_FILES = os.listdir("datasets")


with
    mp_face_detection.FaceDetection( model_selection=1,
    min_detection_confidence=0.5) as
face_detection:
    for idx, file in enumerate(IMAGE_FILES):
        image = cv2.imread(f"datasets/{file}")
        results = face_detection.process(cv2.cvtColor(image,
cv2.COLOR_BGR2RGB))
        if not results.detections:
            print("no faces detected")
            image = np.asarray(image)
            plt.imshow(image)
            plt.show()
            continue
        annotated_image = image.copy()
        for detection in results.detections:
            print('Face Detected')
            image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
            mp_drawing.draw_detection(image, detection)
            image = np.asarray(image)
            plt.imshow(image)
            plt.show()
```

OUTPUT:

```
Face Detected
```




```
no faces detected
```



RESULT:

Thus the implementation of facial recognition application with artificial neural network was executed successfully.

Ex. No.: 04	Study and implement amazon toolkit: Sagemaker
Date:	

AIM:

To study and implement amazon toolkit: Sagemaker.

INTRODUCTION:

Amazon Sagemaker is a fully managed service for data science and machine learning (ML) workflows. You can use Amazon SageMaker to simplify the process of building, training, and deploying ML models.

To train a model, you can include your training script and dependencies in a Docker container that runs your training code. A container provides an effectively isolated environment, ensuring a consistent runtime and reliable training process.

The SageMaker Training Toolkit can be easily added to any Docker container, making it compatible with SageMaker for training models. If you use a prebuilt SageMaker Docker image for training, this library may already be included.

For more information, see the Amazon SageMaker Developer Guide sections on using Docker containers for training

INSTALLATION:

To install this library in your Docker image, add the following line to your Dockerfile:

Run the command

pip3 install sagemaker-training

USAGE:

The following are brief how-to guides. For complete, working examples of custom training containers built with the SageMaker Training Toolkit, please see the example notebooks.

Create a Docker image and train a model

Write a training script (eg. train.py).

Define a container with a Dockerfile that includes the training script and any dependencies.

The training script must be located in the /opt/ml/code directory.
The environment variable SAGEMAKER_PROGRAM defines which file inside the /opt/ml/code directory to use as the training entry point. When training starts, the interpreter executes the entry point defined by SAGEMAKER_PROGRAM. Python and shell scripts are both supported.

FROM yourbaseimage:tag

```
# install the SageMaker Training Toolkit
RUN pip3 install sagemaker-training

# copy the training script inside the container
COPY train.py /opt/ml/code/train.py

# define train.py as the script entry point
ENV SAGEMAKER_PROGRAM train.py
```

Build and tag the Docker image.

```
docker build -t custom-training-container
```

Use the Docker image to start a training job using the SageMaker Python SDK.

```
from sagemaker.estimator import Estimator

estimator =
Estimator(image_name="custom-training-container",
                      role="SageMakerRole",
                      train_instance_count=1,
                      train_instance_type="local")

estimator.fit()
```

To train a model using the image on SageMaker, push the image to ECR and start a SageMaker training job with the image URI.

Pass arguments to the entry point using hyperparameters. Any hyperparameters provided by the training job are passed to the entry point as script arguments. The SageMaker Python SDK uses this feature to pass special hyperparameters to the training job, including sagemaker_program and sagemaker_submit_directory. The complete list of SageMaker hyperparameters is available here.

Implement an argument parser in the entry point script.

For example, in a Python script:

```
import argparse

if___name__== "__main__":
  parser = argparse.ArgumentParser()

  parser.add_argument("--learning-rate", type=int,
default=1)
  parser.add_argument("--batch-size", type=int, default=64)
  parser.add_argument("--communicator", type=str)
  parser.add_argument("--frequency", type=int, default=20)

  args = parser.parse_args()
```

...

Start a training job with hyperparameters.

```
{"HyperParameters": {"batch-size": 256, "learning-rate":
0.0001, "communicator": "pure_nccl"}}
```

Read additional information using environment variables

An entry point often needs additional information not available in hyperparameters. The SageMaker Training Toolkit writes this information as environment variables that are available from within the script. For example, this training job includes the channels training and testing:

```
from sagemaker.pytorch import PyTorch

estimator = PyTorch(entry_point="train.py", ...)

estimator.fit({"training": "s3://bucket/path/to/training/data",
               "testing": "s3://bucket/path/to/testing/data"})
```

The environment variables SM_CHANNEL_TRAINING and SM CHANNEL TESTINGprovide the paths to the channels:

```
import argparse
import os

if __name__== "__main__":
  parser = argparse.ArgumentParser()

...

# reads input channels training and testing from the
environment variables
parser.add_argument("--training", type=str,
default=os.environ["SM_CHANNEL_TRAINING"])
parser.add_argument("--testing", type=str,
```

```
default=os.environ["SM_CHANNEL_TESTING"])

args = parser.parse_args()


...
```

When training starts, SageMaker Training Toolkit will print all available environment variables. Please see the reference on environment variables for a full list of provided environment variables.

Get information about the container environment

To get information about the container environment, initialize an Environment object. The environment provides access to aspects of the environment relevant to training jobs, including hyperparameters, system characteristics, filesystem locations, environment variables, and configuration settings. It is a read-only snapshot of the container environment during training, and it doesn't contain any form of state.

from sagemaker_training import environment

```
env = environment.Environment()

# get the path of the channel "training" from the
`inputdataconfig.json` file
training_dir = env.channel_input_dirs["training"]

# get a the hyperparameter "training_data_file" from
`hyperparameters.json` file
file_name = env.hyperparameters["training_data_file"]

# get the folder where the model should be saved
model_dir = env.model_dir

# train the model
data = np.load(os.path.join(training_dir, file_name))
```

```
x_train, y_train = data["features"],
keras.utils.to_categorical(data["labels"])
model = ResNet50(weights="imagenet")
...
model.fit(x_train, y_train)

#save the model to the model_dir at the end of training
model.save(os.path.join(model_dir, "saved_model"))
```

Execute the entry point

To execute the entry point, call entry_point.run().

from sagemaker_training import entry_point, environment

```
env = environment.Environment()

# read hyperparameters as script arguments
args = env.to_cmd_args()

# get the environment variables
env_vars = env.to_env_vars()

# execute the entry point
entry_point.run(uri=env.module_dir,
                user_entry_point=env.user_entry_point,
                args=args,
                env_vars=env_vars)
```

If the entry point execution fails, trainer.train() will write the error message to /opt/ml/output/failure. Otherwise, it will write to the file /opt/ml/success.

RESULT:

Thus the study and implementation of the amazon toolkit: Sagemaker is done successfully.

Ex. No.: 05	Implement character recognition using Multilayer Perceptron
Date:	

AIM:

To implement character recognition using Multilayer Perceptron.

ALGORITHM:

Step 1: Start the program

Step 2: Import the tensorflow, seaborn, numpy, pandas, matplotlib modules.

Step 3: Load the mnist dataset from keras

Step 4: Create a model using tf.keras.models.sequential method.

Step 5: Set epochs to 5.

Step 6: Test the accuracy of the given test dataset.

Step 7: Plot the heatmap

Step 8: Stop the program

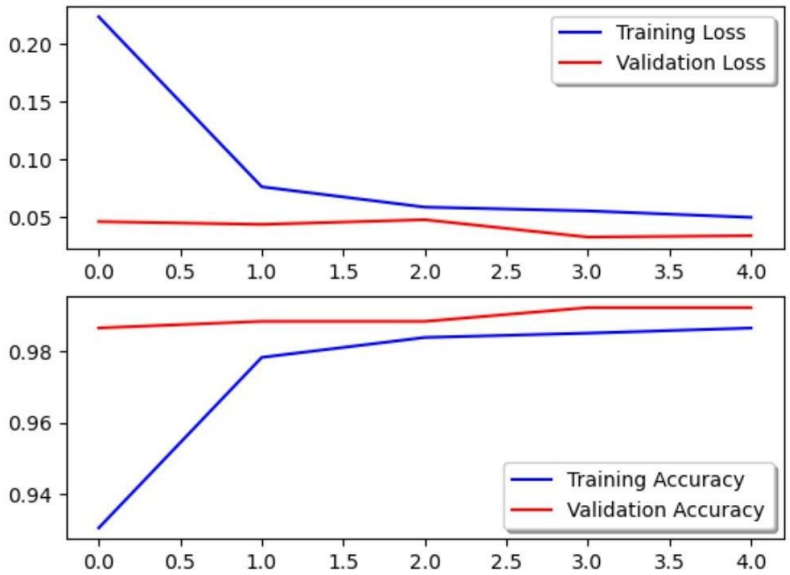
PROGRAM:

```
import tensorflow as tf
import seaborn as sns
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
sns.countplot(y_train)
input_shape = (28, 28, 1)
x_train=x_train.reshape(x_train.shape[0], x_train.shape[1],
x_train.shape[2], 1)
x_train=x_train / 255.0
x_test = x_test.reshape(x_test.shape[0], x_test.shape[1],
x_test.shape[2], 1)
x_test=x_test/255.0
y_train = tf.one_hot(y_train.astype(np.int32), depth=10)
y_test = tf.one_hot(y_test.astype(np.int32), depth=10)
plt.imshow(x_train[100][:,:,0])
print(y_train[100])
batch_size = 64
num_classes = 10
epochs = 5
model =
    tf.keras.models.Sequential([ tf.keras.layers.Conv2D(3
    2, (5,5), padding='same',
activation='relu', input_shape=input_shape),
    tf.keras.layers.Conv2D(32, (5,5), padding='same',
activation='relu'),
    tf.keras.layers.MaxPool2D(),
    tf.keras.layers.Dropout(0.25),
    tf.keras.layers.Conv2D(64, (3,3), padding='same',
activation='relu'),
    tf.keras.layers.Conv2D(64, (3,3), padding='same',
activation='relu'),
    tf.keras.layers.MaxPool2D(strides=(2,2)),
```

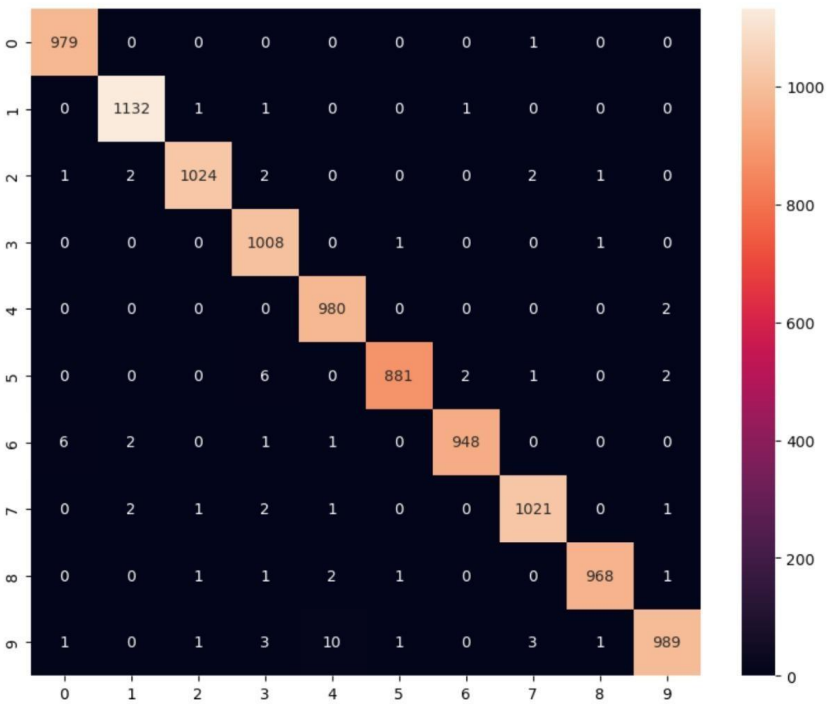
```
    tf.keras.layers.Dropout(0.25),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(num_classes, activation='softmax')
])

model.compile(optimizer=tf.keras.optimizers.RMSprop(epsilon=
1e-08), loss='categorical_crossentropy', metrics=['acc'])
history = model.fit(x_train, y_train,
                    batch_size=batch_size,
                    epochs=epochs,
                    validation_split=0.1)

fig, ax = plt.subplots(2,1)

ax[0].plot(history.history['loss'], color='b',
label="Training Loss")
ax[0].plot(history.history['val_loss'], color='r',
label="Validation Loss",axes =ax[0])
legend = ax[0].legend(loc='best', shadow=True)

ax[1].plot(history.history['acc'], color='b',
label="Training Accuracy")
ax[1].plot(history.history['val_acc'],
color='r',label="Validation Accuracy")
legend = ax[1].legend(loc='best', shadow=True)
test_loss, test_acc = model.evaluate(x_test, y_test)
# Predict the values from the testing dataset
Y_pred = model.predict(x_test)
# Convert predictions classes to one hot vectors
Y_pred_classes = np.argmax(Y_pred,axis = 1)
# Convert testing observations to one hot vectors
Y_true = np.argmax(y_test,axis = 1)
# compute the confusion matrix
confusion_mtx = tf.math.confusion_matrix(Y_true,
Y_pred_classes)
plt.figure(figsize=(10, 8))
sns.heatmap(confusion_mtx, annot=True, fmt='g')
```

OUTPUT:

```
313/313 [==============================] - 10s 31ms/step - loss: 0.0236 - acc: 0.9930
```




```
<AxesSubplot:>
```



RESULT:

Thus the implementation of the concept of character recognition using Multilayer Perceptron Was executed successfully.

Ex. No.: 06	Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs.
Date:	

AIM:

To implement the non-parametric Locally Weighted Regression algorithm in order to fit data points.

ALGORITHM:

Step 1: Start the program

Step 2: Importing the math, NumPy, and scipy modules.

Step 3: Define a function lowess which takes the parameter x, y and f.

Step 4: Set n value equal to 100.

Step 5: x is an array created using random module.

Step 6: y is a sin wave generated using np.sin()

Step 7: Plot the output as a scatter plot using matplotlib.

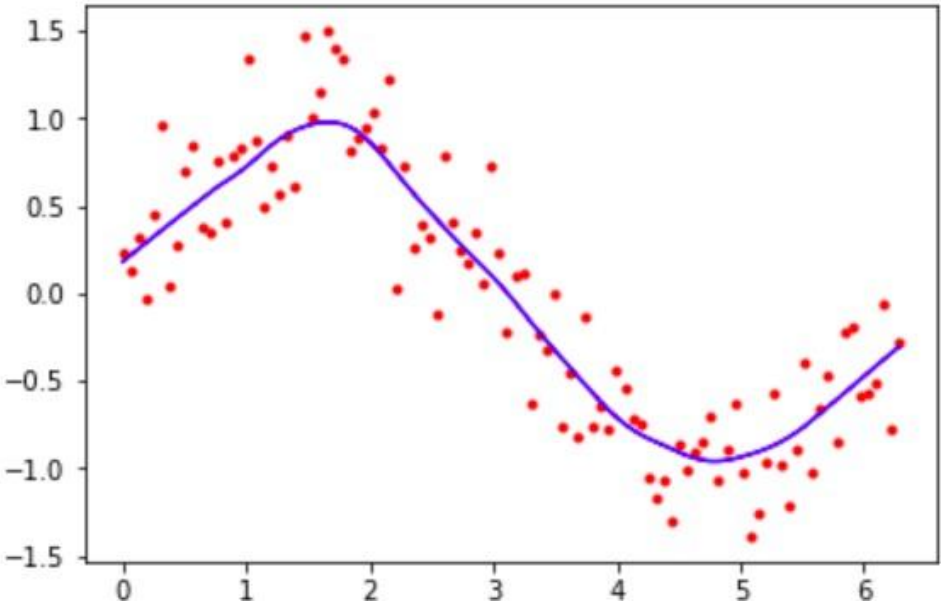
Step 8: Stop the program

PROGRAM:

```
from math import ceil
import numpy as np
from scipy import linalg
def lowess(x, y, f, iterations):
    n = len(x)
    r = int(ceil(f * n))
    h = [np.sort(np.abs(x - x[i]))[r] for i in range(n)]
    w = np.clip(np.abs((x[:,None]-x[None, :])/ h), 0.0, 1.0)
    w = (1 - w ** 3) ** 3
    yest = np.zeros(n)
    delta = np.ones(n)
    for iteration in range(iterations):
        for i in range(n):
            weights = delta * w[:, i]
            b = np.array([np.sum(weights * y),
np.sum(weights * y * x)])
            A = np.array([[np.sum(weights), np.sum(weights *
x)],[np.sum(weights * x), np.sum(weights * x * x)]])
            beta = linalg.solve(A, b)
            yest[i] = beta[0] + beta[1] * x[i]
        residuals = y - yest
        s = np.median(np.abs(residuals))
        delta = np.clip(residuals / (6.0 * s), -1, 1)
        delta = (1 - delta ** 2) ** 2

    return yest #end of func
import math
n = 100
x = np.linspace(0, 2 * math.pi, n)
y = np.sin(x) + 0.3 * np.random.randn(n)
f =0.25
iterations=3
yest = lowess(x, y, f, iterations)
import matplotlib.pyplot as plt
plt.plot(x,y,"r.")
plt.plot(x,yest,"b-")
```

OUTPUT:

```
Out[4]: []
```



RESULT:

Thus the implementation of the non-parametric Locally Weighted Regression algorithm in order to fit data points was successfully executed.

Ex. No.: 07	Implement sentiment analysis using random forest optimization algorithm
Date:	

AIM:

To implement sentiment analysis using random forest optimization algorithm.

ALGORITHM:

Step 1: Start the program

Step 2: Import the Numpy, Pandas, Seaborn, Matplotlib, and nltk modules.

Step 3: Read the dataset using Pandas read_csv().

Step 4: Perform basic EDA using the default summarization methods.

Step 5: Clean the text data using nltk module.

Step 6: Plot a word cloud using the most positive and negative words.

Step 7: Fit the model using RandomForestClassifier.

Step 8: Measure the accuracy using sklearn.metrics.

Step 9: Stop the program.

PROGRAM:

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g.
pd.read_csv)
import seaborn as sns
import matplotlib.pyplot as plt
import re
import nltk
%matplotlib inline

imdb=pd.read_csv('Train.csv')
print(imdb.info())
print(imdb.shape)
print(imdb.head(10))

imdb['label'].value_counts().plot.pie(figsize=(6,6),title="D
istribution of reviews per
sentiment",labels=['',''],autopct='%1.1f%%')
labels=["Positive","Negative"]
plt.legend(labels,loc=3)
plt.gca().set_aspect('equal')

from sklearn.model_selection import train_test_split

features = imdb.drop("label",axis=1)
labels = imdb["label"]

X_train, X_test, y_train, y_test =
train_test_split(features,labels,test_size = 0.90,random_state=42)
X_train, X_test, y_train, y_test =
train_test_split(X_train,y_train,test_size = 0.5,
random_state=42)
X_val, X_test, y_val, y_test =
train_test_split(X_test,y_test,test_size = 0.5,
random_state=42)
```

```
print("Data distribution:\n- Train: {} \n- Validation: {}
\n- Test: {}".format(len(y_train),len(y_val),len(y_test)))

def clean(text):
    wn = nltk.WordNetLemmatizer()
    stopword = nltk.corpus.stopwords.words('english')
    tokens = nltk.word_tokenize(text)
    lower = [word.lower() for word in tokens]
    no_stopwords = [word for word in lower if word not in
stopword]
    no_alpha = [word for word in no_stopwords if
word.isalpha()]
    lemm_text = [wn.lemmatize(word) for word in no_alpha]
    clean_text = lemm_text
    return clean_text

from wordcloud import WordCloud
from collections import Counter

def generate_wordcloud(words,sentiment):
    plt.figure(figsize=(16,13))
    wc = WordCloud(background_color="white", max_words=100,
max_font_size=50)
    wc.generate(words)
    plt.title("Most common {} words".format(sentiment),
fontsize=20)
    plt.imshow(wc.recolor(colormap='Pastel2',
random_state=17), alpha=0.98)
    plt.axis('off')

imdb=imdb.head(1000)
print("Processing data...")
imdb['clean']=imdb['text'].map(clean)
imdb['clean_text']=imdb['clean'].apply(lambda x: "
".join([str(word) for word in x]))

print("Creating word clouds...")
positive_words="
".join(imdb[imdb.label==1]['clean_text'].values)
```

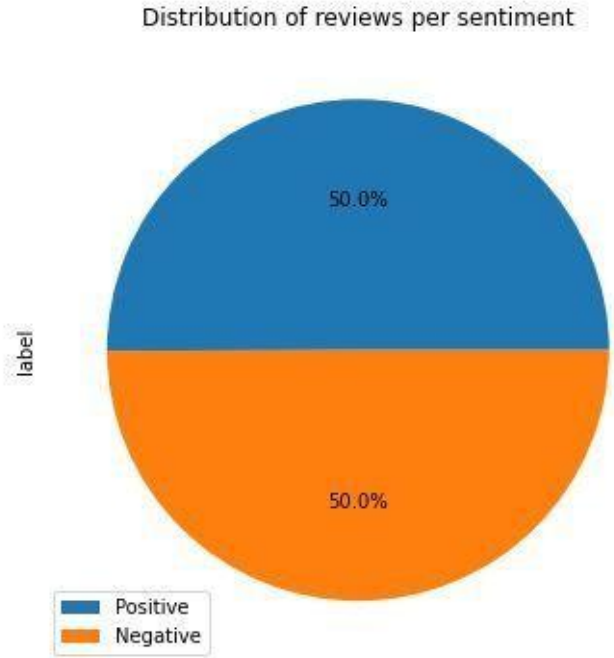
```
negative_words="
".join(imdb[imdb.label==0]['clean_text'].values)

generate_wordcloud(positive_words,"positive")
generate_wordcloud(negative_words,"negative")

from sklearn.feature_extraction.text import TfidfVectorizer

def vectorize(data,tfidf_vect_fit):
    X_tfidf = tfidf_vect_fit.transform(data)
    words = tfidf_vect_fit.get_feature_names()
    X_tfidf_df = pd.DataFrame(X_tfidf.toarray())
    X_tfidf_df.columns = words
    return(X_tfidf_df)

tfidf_vect = TfidfVectorizer(analyzer=clean)
tfidf_vect_fit=tfidf_vect.fit(X_train['text'])
X_train=vectorize(X_train['text'],tfidf_vect_fit)

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

rf = RandomForestClassifier()
scores =
cross_val_score(rf,X_train,y_train.values.ravel(),cv=5)

print(scores)
scores.mean()

def print_results(results):
    print('BEST PARAMS: {}\n'.format(results.best_params_))

    means = results.cv_results_['mean_test_score']
    stds = results.cv_results_['std_test_score']
    for mean, std, params in zip(means, stds,
results.cv_results_['params']):
        print('{} (+/-{}) for {}'.format(round(mean, 3),
round(std * 2, 3), params))
```

```
from sklearn.model_selection import GridSearchCV

rf = RandomForestClassifier()
parameters = {
    'n_estimators': [5,50,100],
    'max_depth': [2,10,20,None]
}

cv = GridSearchCV(rf,parameters)
cv.fit(X_train,y_train.values.ravel())
print_results(cv)

X_val=vectorize(X_val['text'],tfidf_vect_fit)

rf1 = RandomForestClassifier(n_estimators=100,max_depth=20)
rf1.fit(X_train, y_train.values.ravel())
rf2 =
RandomForestClassifier(n_estimators=100,max_depth=None)
rf2.fit(X_train, y_train.values.ravel())
rf3 = RandomForestClassifier(n_estimators=5,max_depth=None)
rf3.fit(X_train, y_train.values.ravel())

from sklearn.metrics import
accuracy_score,precision_score,recall_score

for mdl in [rf1,rf2,rf3]:
    y_pred = mdl.predict(X_val)
    accuracy = round(accuracy_score(y_val,y_pred), 3)
    precision = round(precision_score(y_val,y_pred), 3)
    recall = round(recall_score(y_val,y_pred), 3)
    print('MAX DEPTH: {} / # OF EST: {} -- A: {} / P: {} /
R: {}'.format(mdl.max_depth,

mdl.n_estimators,

accuracy,

precision,
```

```
recall))

X_test=vectorize(X_test['text'],tfidf_vect_fit)

y_pred = rf2.predict(X_test)
accuracy = round(accuracy_score(y_test,y_pred), 3)
precision = round(precision_score(y_test,y_pred), 3)
recall = round(recall_score(y_test,y_pred),  3)
print('MAX DEPTH: {} / # OF EST: {} -- A: {} / P: {} / R:
{}'.format(rf3.max_depth,

rf3.n_estimators,

accuracy,

precision,

recall))
feat_importances = pd.Series(rf2.feature_importances_,
index=X_train.columns)
feat_importances.nlargest(20).plot(kind='bar',figsize=(10,10
))
plt.title("Top 20 important features")
plt.show()
```

OUTPUT:

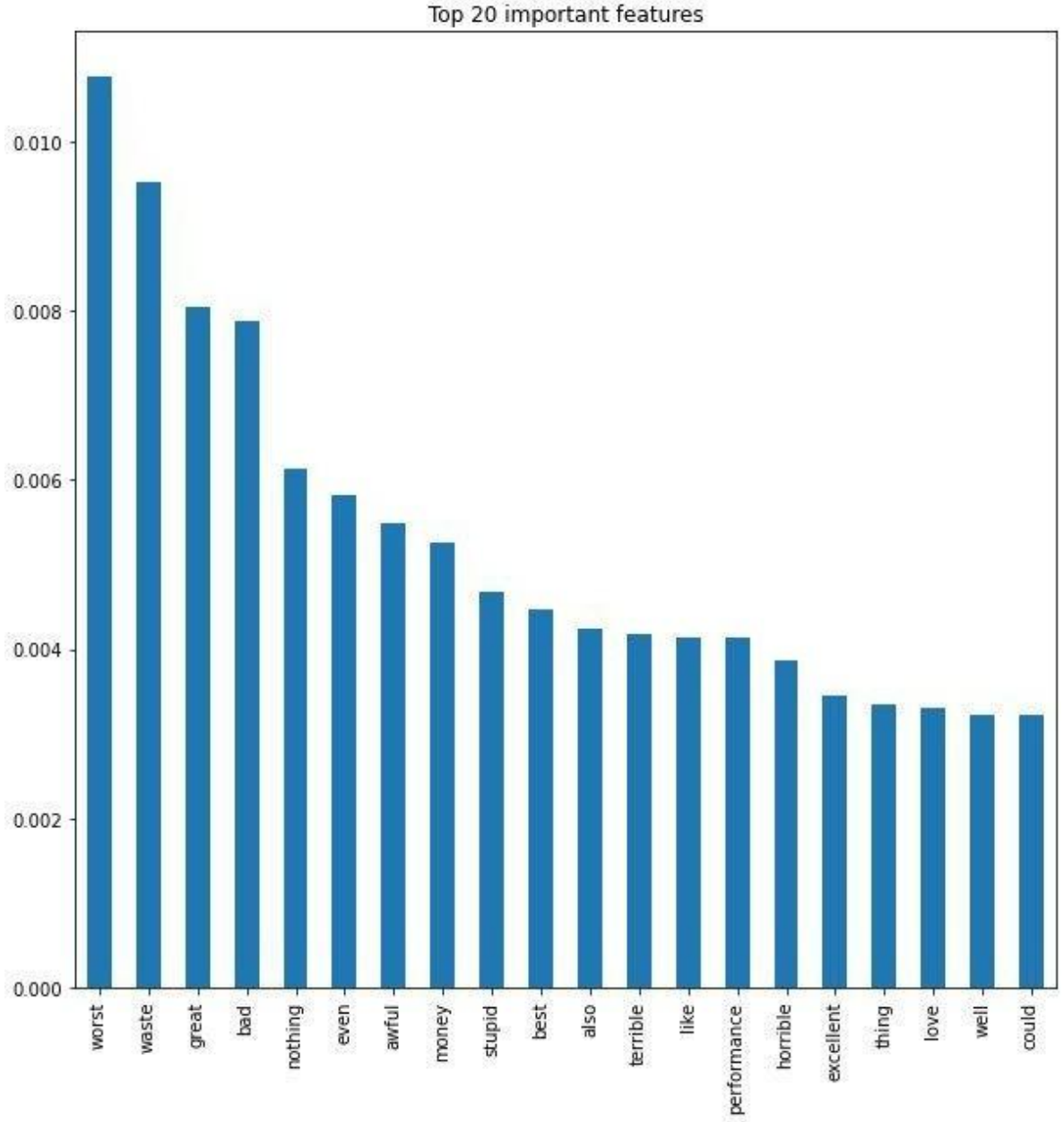
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 40000 entries, 0 to 39999
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   text    40000 non-null  object
 1   label   40000 non-null  int64
dtypes: int64(1), object(1)
memory usage: 625.1+ KB
None
(40000, 2)
                                                text  label
0  I grew up (b. 1965) watching and loving the Th...      0
1  When I put this movie in my DVD player, and sa...      0
2  Why do people who do not know what a particula...      0
3  Even though I have great interest in Biblical ...      0
4  Im a die hard Dads Army fan and nothing will e...      1
5  A terrible movie as everyone has said. What ma...      0
6  Finally watched this shocking movie last night...      1
7  I caught this film on AZN on cable. It sounded...      0
8  It may be the remake of 1987 Autumn's Tale aft...      1
9  My Super Ex Girlfriend turned out to be a plea...      1
```



```
Processing data...
Creating word clouds...
```

Most common positive words

Most common negative words


Top 20 important features
0.010
0.008
0.006
0.004
0.002
0.000
worst
waste
great
bad
nothing
even
awful
money
stupid
best
also
terrible
like
performance
horrible
excellent
thing
love
well
could

RESULT:

Thus the implementation of sentiment analysis using random forest optimization algorithm was executed successfully.

Ex. No.: 08	Write a program to construct a Bayesian network using standard Heart Disease Data Set. You can use Java/Python ML library classes/API.
Date:	

AIM:

To implement the concept of Bayesian network using a standard Heart Disease DataSet.

ALGORITHM:

Step 1: Start the program

Step 2: Importing the necessary modules.

Step 3: Read the dataset using pandas read_csv() method.

Step 4: Split the test and train dataset.

Step 5: convert it into a vector.

Step 6: Fit the model using SVC.fit()

Step 7: Print the score of the classifier.

Step 8: Stop the program

PROGRAM:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn as skl
heart = pd.read_csv('heart.csv')
heart.head()
heart.tail()
heart.shape
heart.isnull().sum()

!pip install pgmpy
from pgmpy.models import BayesianModel
from pgmpy.estimators import MaximumLikelihoodEstimator,
BayesianEstimator
heart.columns

model =
BayesianModel({('Age','ChestPainType'),('Age','RestingBP'),(
'Sex','ChestPainType'),('Sex','RestingBP'),('ChestPainType',
'HeartDisease'),('RestingBP','HeartDisease')})
model.fit(heart,estimator = MaximumLikelihoodEstimator)
print(model.get_cpds('Age'))
```

OUTPUT:

Age(59)	0.0381264
Age(60)	0.0348584
Age(61)	0.0337691
Age(62)	0.0381264
Age(63)	0.0326797
Age(64)	0.0239651
Age(65)	0.0228758
Age(66)	0.0141612
Age(67)	0.0163399
Age(68)	0.0108932
Age(69)	0.0141612
Age(70)	0.00762527
Age(71)	0.00544662
Age(72)	0.0043573
Age(73)	0.00108932
Age(74)	0.00762527
Age(75)	0.00326797
Age(76)	0.00217865

RESULT:

Thus the implementation of the concept of Bayesian network using a standard Heart disease dataset was executed successfully.

Ex. No.: 09	Choose best machine learning algorithm to implement online fraud detection
Date:	

AIM:

To select the best machine learning algorithm to implement online fraud detection.

ALGORITHM:

Step 1: Start the program

Step 2: Import the necessary modules.

Step 3: Perform basic EDA using the default summarization methods.

Step 4: Allocate the variable of the dataset

Step 5: Split the dataset using train_test_split method in model_selection module.

Step 6: Fit the LogisticRegression model for the train dataset.

Step 7: Predict the value using LogisticRegression model.

Step 8: Fit the DecisionTreeClassifier model for the train dataset.

Step 9: Predict the value using DecisionTreeClassifier model.

Step 10: Fit RandomForestClassifier model for the train dataset

Step 11: Predict the value using RandomForestClassifier model.

Step 12: Calculate the accuracy for all the three models and create a DataFrame.

Step 13: Plot the DataFrame using Seaborn.barplot method.

Step 14: Stop the program.

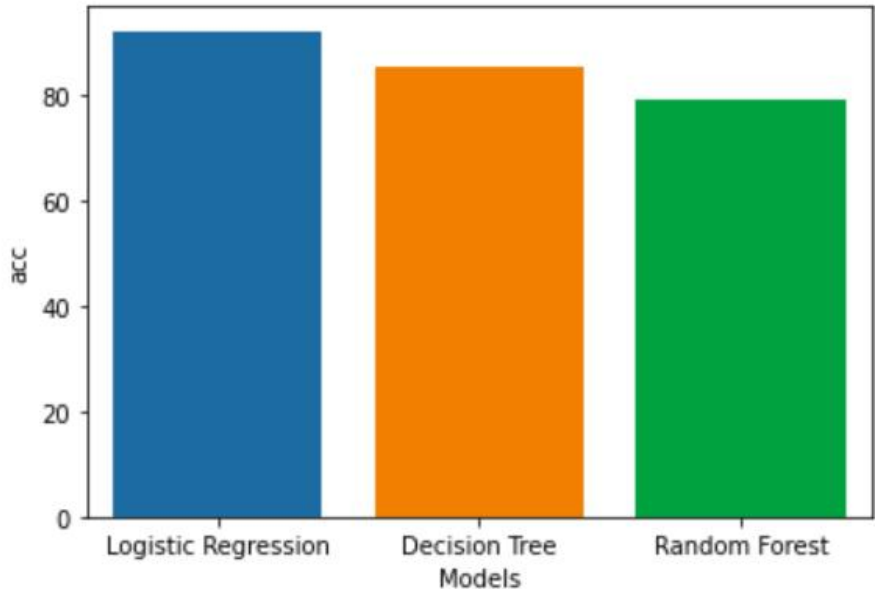
PROGRAM:

```
import pandas as pd
data = pd.read_csv('creditcard.csv')
data.head()
data.tail()
data.shape
data.info()
data.isnull().sum()
data.head()
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
data['Amount'] =
sc.fit_transform(pd.DataFrame(data['Amount']))
data.head()
data = data.drop(['True'],axis=1)
data.head()
data.duplicated().any()
data = data.drop_duplicates()
data.shape
data['class'].value_counts()
x = data.drop('class',axis=1)
y = data['class']
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test =
train_test_split(x,y,test_size=0.20,random_state=42)

from sklearn.linear_model import LogisticRegression
log =  LogisticRegression()
log.fit(x_train,y_train)
y_pred1 = log.predict(x_test)
from sklearn.metrics import accuracy_score
accuracy_score(y_test,y_pred1)
from sklearn.tree import DecisionTreeClassifier
dt = DecisionTreeClassifier()
dt.fit(x_train,y_train)
y_pred2 = log.predict(x_test)
accuracy_score(y_test,y_pred2)
```

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier()
rf.fit(x_train,y_train)
y_pred3 = log.predict(x_test)
accuracy_score(y_test,y_pred3)
final_data = pd.DataFrame({'Models':['lr','dt','rf'],

"acc":[accuracy_score(y_test,y_pred1)*100,

accuracy_score(y_test,y_pred2)*100,

accuracy_score(y_test,y_pred3)*100]})
import seaborn as sns
sns.barplot(final_data[‘Model’],final_data[‘acc’])
sns.barplot(data=final_data, x="Models", y="acc")
```

OUTPUT:

```
Out[5]: <AxesSubplot:xlabel='Models', ylabel='acc'>
```



RESULT:

Thus the best machine learning algorithm to implement online fraud detection has been identified successfully.